

HumanEd: MV2 "Morris" 6-Axis Serial Manipulator

Benjamin McEwan

October 10, 2026

Contents

1	Overview	2
2	Kinematics	3
2.1	Forward Kinematics	4
2.2	Workspace & Joint Limits	5
3	Electronics & Software	6
3.1	UNO R3 Setup	7
4	Assembly	8

1 Overview



Figure 1: The CAD model of MV2.

The MV2 robot has been designed as a low-cost 6-axis serial manipulator that can be easily tailored to use different control systems and end-effectors. The robot follows the standard joint arrangement seen on many industrial 6-DOF manipulators, with a 2R open-chain attached to a rotating base used as the positioning system for a spherical wrist.

2 Kinematics

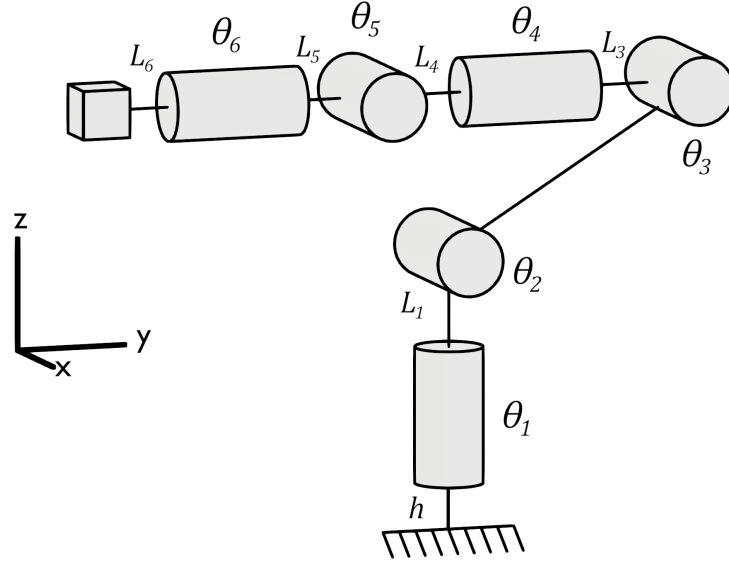


Figure 2: Joint diagram of the MV2 robot.

Figure 2 shows the joints of the MV2 robot. θ_1 , θ_2 , and θ_3 comprise the positioning joints; θ_4 , θ_5 , and θ_6 comprise the orienting joints, constructed in the spherical wrist configuration.

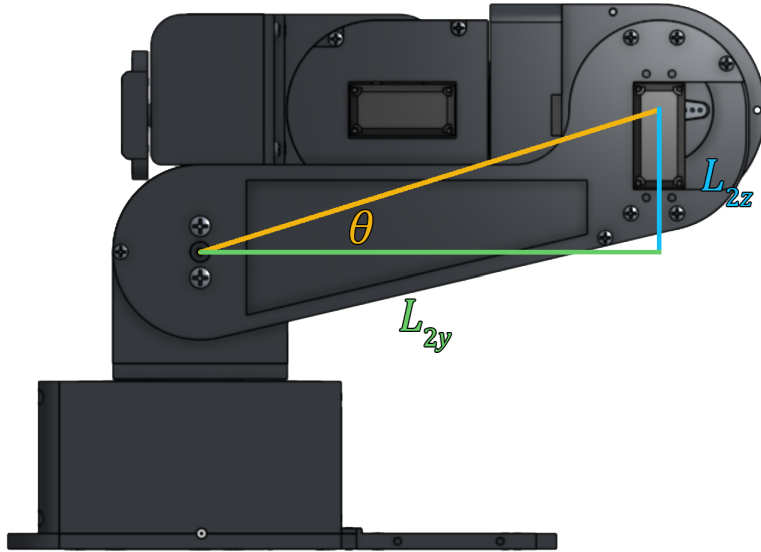


Figure 3: Diagram of y -axis and z -axis offset distances between joint 2 and joint 3.

The only notable deviation from this structure on the MV2 is that when the robot is in its rest position, there is an angled offset between joint 2 and joint 3. This can be seen in Figure 3,

which shows a side view of MV2 with the aesthetic shell of its second link removed. The offset has been marked with distances L_{2y} and L_{2z} , and θ as the resulting offset angle. As such, any kinematic models of the robot must take into account this offset. All other joints are collinear and/or coaxial where relevant.

2.1 Forward Kinematics

The forward kinematics of MV2 can be defined using the Power of Exponentials formula discussed by Lynch and Park in Modern Robotics, [1].

As per Figure 2, define:

- h as the distance between the ground and θ_1 ;
- L_1 as the distance between θ_1 and θ_2 ;
- L_3 as the distance between θ_3 and θ_4 ;
- L_4 as the distance between θ_4 and θ_5 ;
- L_5 as the distance between θ_5 and θ_6 ;
- and L_6 as the distance between θ_6 the end-effector.

L_2 is defined as the components L_{2y} and L_{2z} , as per Figure 3. Screw axes $\mathcal{S}_n$ can then be defined for each joint as follows:

$$\mathcal{S}_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ h \end{bmatrix}, \quad \mathcal{S}_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ h + L_1 \end{bmatrix}, \quad \mathcal{S}_3 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ -L_{2y} \\ h + L_1 + L_{2z} \end{bmatrix}$$

$$\mathcal{S}_4 = \begin{bmatrix} 0 \\ -1 \\ 0 \\ 0 \\ -L_{2y} + L_3 \\ h + L_1 + L_{2z} \end{bmatrix}, \quad \mathcal{S}_5 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ -L_{2y} + L_3 + L_4 \\ h + L_1 + L_{2z} \end{bmatrix}, \quad \mathcal{S}_6 = \begin{bmatrix} 0 \\ -1 \\ 0 \\ 0 \\ -L_{2y} + L_3 + L_4 + L_5 \\ h + L_1 + L_{2z} \end{bmatrix}$$

and the end-effector transformation at rest, M can be defined as

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -L_{2y} + L_3 + L_4 + L_5 + L_6 \\ 0 & 0 & 1 & h + L_1 + L_{2z} \\ 0 & 0 & 0 & 1 \end{bmatrix} \in SE(3)$$

which can be used in the PoE formula:

$$T(\theta) = e^{[\mathcal{S}_1]\theta_1} e^{[\mathcal{S}_2]\theta_2} e^{[\mathcal{S}_3]\theta_3} e^{[\mathcal{S}_4]\theta_4} e^{[\mathcal{S}_5]\theta_5} e^{[\mathcal{S}_6]\theta_6} M \in SE(3), \quad (1)$$

$$\theta = [\theta_1 \quad \theta_2 \quad \theta_3 \quad \theta_4 \quad \theta_5 \quad \theta_6]^T \in \mathbb{R}^6$$

Table 1 shows the physical values of the parameters used in Equation 1.

Table 1: Physical values for PoE parameters.

Parameter	Value (mm)
h	65.35
L_1	50.30
L_{2y}	179.00
L_{2z}	55.30
L_3	65.85
L_4	44.30
L_5	88.00
L_6	(Decided by chosen end-effector)

Table 2: Joint angle limits, listed with reference axis taken as 0° .

Joint	Reference Axis	Limit ($^\circ$)	Rest Position ($^\circ$)
θ_1	x	180° ¹	90°
θ_2	y	180° ¹	0°
θ_3	$-y$	180°	0°
θ_4	$-x$	180°	90°
θ_5	$-z$	180°	90°
θ_6	x	180°	90°

2.2 Workspace & Joint Limits

As the robot is constructed using servos for actuation, the joints are not continuous and have specific limits. These can be seen in Table 2. The reference axes and subsequent rest positions for each joint are based on the servos themselves rather than the mathematical model of Equation 1. The reference axes are defined as per the axes shown in Figure 2.

¹Technically, θ_1 and θ_2 both use the Miuzei MS24 20kg Digital Servo, which has an angle range of 270° , however for consistency, ease of use, and to simplify the construction, the servo has simply been taken as having the same 180° limit as the MG996R servos used in the other joints.

3 Electronics & Software

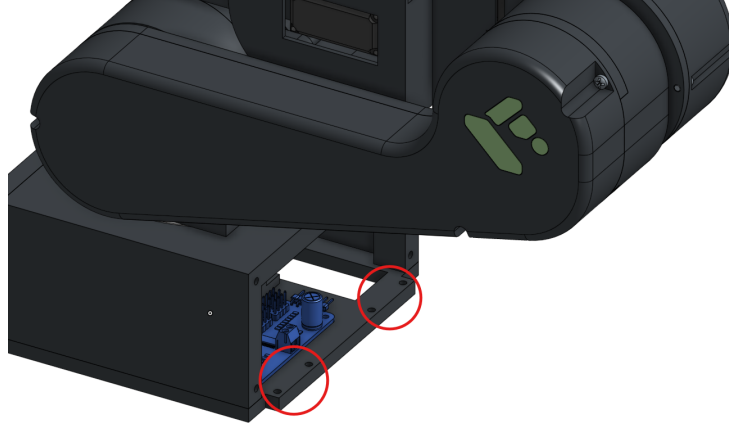


Figure 4: Positions of the electronics mounting holes.

MV2 has been designed such that it is not tied directly to one microcontroller platform. The base of MV2 has been designed with an open back, and four mounting holes, as seen in Figure 4, which can be used to fit custom 3D printed baseplate extensions to hold whichever microcontroller and power boards the user may wish to use.

The only electronics contained within the base link (aside from the joint 1 motor) is an Adafruit-style PCA9685 16-Channel 12-Bit PWM servo driver board. Mounting holes have been included for this board as it was considered the most basic element needed to control the robot, however should they wish, the open design of the base means that the user is free to forego this board in favour of an alternative.

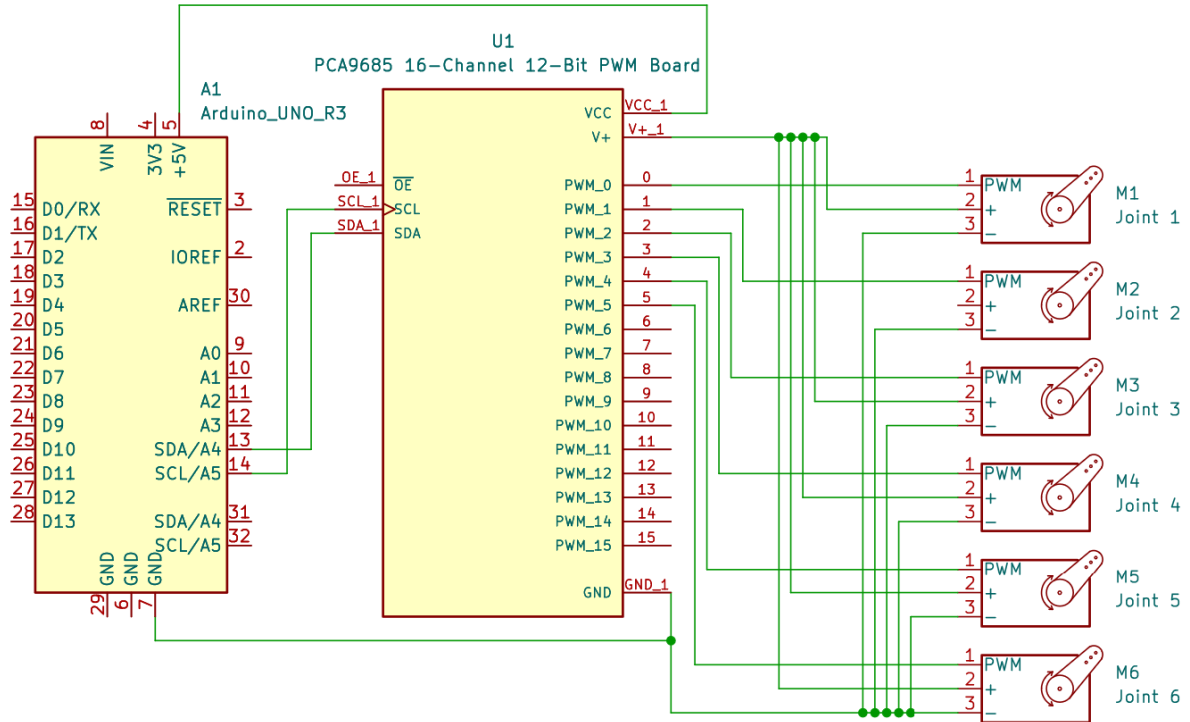


Figure 5: Schematic of the MV2 UNO-R3 based electronics.

3.1 UNO R3 Setup

As a basic "default" setup, the MV2 repository includes a mount and control script for controlling the robot using an Arduino UNO R3 microcontroller. The formal schematic for this is shown in Figure 5, however it may also be clearer to some users to simply state the connections, which are as follows:

- UNO R3: 5V →PCA9685: VCC
- UNO R3: GND →PCA9685: GND
- UNO R3: SCL →PCA9685: SCL
- UNO R3: SDA →PCA9685: SDA
- Motors 1, 2, 3, 4, 5, 6 to PCA9685 channels 0, 1, 2, 3, 4, 5 respectively.

Note that this setup assumes the UNO R3 model that includes dedicated SDA and SCL pins for I2C.

A script is included in the UNO R3 setup, in the form of an Arduino IDE sketch named `mv2_control_uno-r3.ino`. This script allows the MV2 robot to be controlled using commands over serial, and makes use of acceleration ramps in order to reduce oscillations in the robot's movements. The script has three commands, which are:

- `joint [channel] [target angle] [time to move in milliseconds]`

Tells a specific joint motor (based on its PCA9685 channel) to move to a target angle in some set amount of time.

Example: `joint 2 60 1500`

The above will tell the elbow joint to move to 60° in 1.5 seconds. It is recommended to try and keep movement times to a minimum of 1000 milliseconds.

- `home [time to move in milliseconds]`

Tells the robot to move to its rest, or home, position in some set amount of time.

Example: `home 3000`

The above will tell the robot to move to its rest position in 3 seconds. It is recommended that when writing serial control code (for example, in Python) that you use this command as the exit state of your script to avoid the robot jerking into its rest position each time you reset the control script.

- `stance [time to move in milliseconds]`

Tells the robot to move to its "stance" position in some set amount of time.

Example: `stance 2000`

The above will tell the robot to move to its rest position in 2 seconds. This can be a useful command for verifying that the robot's joints are working correctly.

4 Assembly

MV2 is a 3D printed robot. The robot has been designed to be printed using PLA at 10% to 20% infill, however any reasonably hard filament should function as well. The `.stl` files for MV2 can be found in the `/parts/` folder of the MV2 repository. Assembly drawings can be found in the `.pdf` file named `mv2-drawings_assembly.pdf`.

Aside from 3D printed parts, MV2 uses various purchased components. The list of these can be seen in Table [3](#)

When assembling motor and horn stabiliser pairs, the Arduino IDE sketch named `mv2_servo-zeroing.ino` should be used to ensure that all servos are in their correct home positions when the links are all assembled together.

Table 3: Bill of Materials

Part Description	Type	Quantity
Base Attachment	3D Printed	1
Base Servo Mount	3D Printed	1
Baseplate	3D Printed	1
End Effector Mount	3D Printed	1
Frontplate	3D Printed	1
Horn Stabiliser (Link 1)	3D Printed	1
Horn Stabiliser (MG996R)	3D Printed	3
Horn Stabiliser (MS24)	3D Printed	1
Link 1 Attachment	3D Printed	1
Link 1 Base	3D Printed	1
Link 1 Servo Mount	3D Printed	1
Link 1 Shell	3D Printed	1
Link 2 Attachment	3D Printed	1
Link 2 Servo Mount	3D Printed	1
Link 2 Shell	3D Printed	1
Link 3 Attachment	3D Printed	1
Link 3 Servo Mount	3D Printed	1
Link 3 Shell	3D Printed	1
Link 4 Attachment	3D Printed	1
Link 4 Servo Mount	3D Printed	1
Link 4 Shell	3D Printed	1
Link 5 Attachment	3D Printed	1
Link 5 Servo Mount	3D Printed	1
Link 5 Shell	3D Printed	1
Logo 1	3D Printed	1
Logo 2	3D Printed	1
Logo 3	3D Printed	1
Logo 4	3D Printed	1
6806 Bearing 42x30x7mm	Purchased	4
6815 Bearing 95x75x10mm	Purchased	1
MG996 Servo Motor	Purchased	4
MG996 Straight Horn	Purchased	6
MS24 Servo (Miuzei 20kg)	Purchased	2
PCA9685 16-Channel 12-Bit PWM Servo Driver Board	Purchased	1
M3 Stainless Steel Washer	Purchased	16
Pan head screw M3 x 10 Stainless Steel	Purchased	41
Pan head screw M3 x 12 Stainless Steel	Purchased	12
Pan head screw M3 x 6 Stainless Steel	Purchased	9
Pan head screw M3 x 8 Stainless Steel	Purchased	6
Pan head screw M4 x 10 Stainless Steel	Purchased	18
Pan head screw M4 x 20 Stainless Steel	Purchased	2

References

- [1] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.